

2026-06-10-blocked-cli-agents

Research — Blocked CLI Agents: Headless / Non-Interactive Invocation

Date: 2026-06-10 **Author:** DEEP-WEB-RESEARCH subagent (§11.4.99 latest-source cross-reference) **Scope:** HelixCode `helix_code/cmd/cli/main.go` CLI-dispatch agents left research-blocked after the SP4-cont de-bluff pass (43 dispatch methods: 3 converted to real exec, 40 to honest-error). This doc determines, per agent, whether a CURRENT official non-interactive single-prompt invocation + machine-readable model listing exists, so honest-error stubs can be promoted to real exec. **Method:** WebSearch + WebFetch against official docs / repos as of 2026-06-10. Every finding cited. Honest negatives stated explicitly. No code edited.

TL;DR verdict table

#	Agent	Headless single-prompt invocation?	Verdict	Convertible to real exec?
1	Amazon Q Developer CLI → Kiro CLI	YES — <code>kiro-cli chat --no-interactive "<prompt>"</code>	CONFIRMED HEADLESS	YES
2	Cody (Sourcegraph)	YES — <code>cody chat -m '<prompt>'</code> (officially “human-interactive only”, caveat)	CONFIRMED HEADLESS (with caveat)	YES (best-effort)
3	Sourcegraph <code>src</code> CLI	N/A — search/admin tool, no LLM chat	NO LLM CLI (out of scope)	NO — not an LLM agent
4	SWE-agent	YES — <code>sweagent run --problem_statement.text="..."</code>	CONFIRMED HEADLESS	YES
5	GPT-Pilot / Pythagora	NO — interactive-by-design, no headless flag	NO HEADLESS CLI	NO — stays honest-error
6	Devika	NO — web-UI + socket-driven; no documented single-prompt headless path	NO HEADLESS CLI	NO — stays honest-error
7	Model group (LlamaCode/ StarCoder/ CodeGemma/ DeepSeekCoder/ MistralCode/ Codey/ WizardCoder)	These are MODELS, not CLIs	MODELS, not agents	Via Ollama / provider API — see §7

1. Amazon Q Developer CLI → Kiro CLI

Major finding (currency): The open-source `aws/amazon-q-developer-cli` repo is **no longer actively maintained** (“will only receive critical security fixes”). Amazon Q Developer CLI is now shipped as **Kiro CLI**, a closed-source product, since 2025-11-17. The binary name changed from `q` to `kiro-cli`. Functionally the chat experience is the same. HelixCode should target the Kiro CLI name (and keep `q` as a legacy fallback).

Confirmed non-interactive invocation (Kiro CLI):

```
# Auth: export KIRO_API_KEY=<key>    (headless mode requires the
env var)
kiro-cli chat --no-interactive "your prompt here"
kiro-cli chat --no-interactive --trust-all-tools "Show me the
current directory"
```

- `--no-interactive` — “Print first response to STDOUT without interactive mode” (prints response and exits; an initial prompt arg is mandatory).
- `--trust-all-tools` — auto-approve all tool calls (needed for unattended runs).
- `--trust-tools=<read, grep, write>` — auto-approve specific tool categories.
- `--require-mcp-startup` — exit code 3 if any MCP server fails to start.

Model listing (machine-readable):

```
kiro-cli chat --list-models --format json
```

- `--list-models` — “Display available models”; supports `--format json`. This satisfies the LLMsVerifier-style real model-listing need (CONST-036/037) for a per-agent query — though note Kiro’s model set is its own, not arbitrary.

Note on legacy `q`: The older `q chat --no-interactive "<prompt>"` form was documented for the AWS-branded CLI and a community request for non-interactive mode (Issue #808) drove it. If an operator still has the `q` binary installed, the same `--no-interactive` flag applies.

Verdict: CONFIRMED HEADLESS — convertible to real exec. Dispatch should prefer `kiro-cli`, fall back to `q`, require `KIRO_API_KEY` (or existing Q auth), pass `--no-interactive --trust-all-tools`, and use `--list-models --format json` for model enumeration.

2. Cody (Sourcegraph) — headless CLI exists, with an official caveat

Confirmed non-interactive invocation:

```
cody chat -m 'Explain React hooks'
cody chat --message 'Explain React hooks'
cody chat Explain React hooks          # space-separated bare
args
cat error.log | cody chat 'What went wrong?' # piped stdin
cody chat --model 'claude-3.5-sonnet' -m 'Hi Cody!'
```

Documented flags (from the official Install Cody CLI page): - m / --message — the prompt text. - --stdin — read message from standard input. - --context-file — local-file context. - --context-repo — remote-repo context. - --model — select model (shown in Sourcegraph blog/docs examples, e.g. --model 'claude-3.5-sonnet').

Caveats (honest): - Sourcegraph docs state the Cody CLI is “**only intended for human interactive usage**” and is “**Experimental**” for Enterprise accounts. It runs single prompts fine in scripts, but Sourcegraph does not formally support automation, and behaviour may change. - **No documented JSON output flag** and **no documented model-listing subcommand** were found in the official CLI docs. Model selection is by passing a known model id to --model; there is no confirmed cody models discovery command. (Negative finding.) - Package: @sourcegraph/cody on npm (the npm page returned 403 to the fetcher; flags above are from the official docs page).

Verdict: CONFIRMED HEADLESS (with caveat) — convertible to real exec (best-effort). Wire cody chat --stdin (or -m) for the prompt; treat model-listing as unavailable (cannot satisfy a real GetModels() from the CLI — would have to hardcode/omit, so list-models for Cody should remain honest-error or defer to Sourcegraph config). Flag the “experimental / human-interactive only” status in the integration doc.

3. Sourcegraph src CLI — not an LLM agent

Finding: The src CLI (sourcegraph/src-cli, npm @sourcegraph/src) is a **code-search / batch-changes / GraphQL-admin** tool. It has **no built-in LLM chat or completion**. It does produce JSON for programmatic consumption (src search -json, raw GraphQL via src api), but that is code search, not AI generation. Sourcegraph’s AI surface is **Cody** (§2), a separate binary.

Verdict: NO LLM CLI — out of scope as an “AI agent”. If HelixCode lists src as an AI agent that is a categorization error: it should either be removed from the AI-agent dispatch table or kept as honest-error with the note “src is a code-search CLI, not an LLM agent; use Cody (§2) for AI.” No conversion to AI real-exec is possible.

4. SWE-agent — headless run-once confirmed

Confirmed single-task invocation:

```
# Single task from inline text:
sweagent run \
  --agent.model.name=claude-3-7-sonnet-latest \
  --problem_statement.text="Fix the bug in foo.py"

# From a GitHub issue / file:
sweagent run --problem_statement.type=github_issue --
  problem_statement.github_url=<url>
sweagent run --config agent.yaml --config env.yaml --
  problem_statement.text="..."

# Batch:
```

```
sweagent run-batch ...
sweagent run-replay <trajectory.json>
```

- `sweagent run` is the headless single-issue entrypoint. It builds a config object; the problem statement may be `TextProblemStatement` (inline), `GithubIssue`, or `FileProblemStatement`. Model is set via `--agent.model.name=...` (LiteLLM-style model ids).
- Output is a trajectory file (machine-readable JSON), not a single text blob — so `HelixCode` would parse the resulting patch/trajectory rather than read a chat response off stdout.
- A lighter sibling exists: **mini-swe-agent** (`mini / mini-swe-agent`), a ~100-line headless agent scoring >74% on SWE-bench verified, also runnable headlessly.

Model listing: SWE-agent has no “list models” command of its own; it delegates to LiteLLM/provider model ids. Model enumeration would come from the underlying provider, not SWE-agent. (Negative for a native list-models.)

Verdict: CONFIRMED HEADLESS — convertible to real exec. Wire `sweagent run --problem_statement.text=<prompt> --agent.model.name=<model>`; consume the emitted trajectory/patch as the result. It is a task-solving harness (issue→patch), not a chat-completion endpoint — the `HelixCode` integration should treat its “response” as the produced patch/trajectory, not free-form text.

5. GPT-Pilot / Pythagora — interactive by design, NO headless

Finding: GPT-Pilot is invoked as `python main.py` (no `console_scripts` entrypoint on PyPI). Args exist (`app_id=`, `workspace=`, `advanced=True`) but the tool is **fundamentally interactive**: the `ProductOwner` agent prompts for app type/name if not given, asks the developer to review each finished task, and requests approval before running commands. There is **no documented non-interactive / headless flag**. Automating it fully would require source modification or expect-style wrapper scripting around prompts — not a supported invocation.

Verdict: NO HEADLESS CLI — stays honest-error. Honest-error message should say: “GPT-Pilot has no non-interactive mode; it requires developer review/approval at each step (run via `python main.py`). Not invocable headlessly.” No clean conversion.

6. Devika — web-UI + socket-driven, NO documented headless single-prompt path

Finding: Devika runs as a Flask backend (`python devika.py` → “Devika is up and running!”) plus a separate SvelteKit/Node UI (`bun run start` at `http://127.0.0.1:3001`). The backend `devika.py` imports Flask (`Flask`, `request`, `jsonify`, `send_file`) and registers blueprints, and there ARE `/api/*` routes (e.g. `/api/get-agent-state`, `/api/get-browser-snapshot`, and a project blueprint). **However:** the README/architecture docs document **only the browser workflow** (create project in UI, set model, type objective in chat). There is **no documented CLI** and **no documented single-prompt “execute task” HTTP endpoint** suitable for clean scriptable invocation; execution is orchestrated through the UI + socket events, and the project is effectively unmaintained. Reverse-engineering an undocumented `/api/execute`-style route would be guessing (forbidden under §11.4.6) and brittle.

Verdict: NO HEADLESS CLI — stays honest-error. Honest-error message: “Devika is a web-UI + socket-driven agent (python devika.py + browser UI); no documented non-interactive CLI or single-prompt API. Not invocable headlessly.” If a future operator wants it, the path would be an undocumented Flask-API integration (research-blocked, not CLI).

7. Model-not-CLI group — LlamaCode, StarCoder, CodeGemma, DeepSeekCoder, MistralCode, Codey, WizardCoder

Finding (confirmation): These are **LLM models / model families, not standalone CLI agents**. None ships an official per-agent headless CLI binary. They are served, not invoked: - **StarCoder, CodeGemma, DeepSeek-Coder, WizardCoder, “LlamaCode”-class** → open-weight code models distributed via Hugging Face, runnable locally through **Ollama** (or llama.cpp / vLLM / HF Transformers). - **Codey** → Google’s code model, served via the **Vertex AI / Gemini provider API** (no local CLI; superseded by Gemini code models). - **MistralCode** → Mistral’s code offering, served via the **Mistral provider API** (and some weights via Ollama/HF).

Recommended HelixCode path — via Ollama (confirmed) or provider API, NOT a per-agent CLI:

```
# Single non-interactive prompt → stdout (confirmed official syntax):
```

```
ollama run <model> "<prompt>"
```

```
ollama run deepseek-coder "Write a quicksort in Go"
```

```
ollama run starcoder2 "Explain this regex" > out.txt #  
redirect/pipe OK
```

```
# Machine-readable model listing (confirmed):
```

```
ollama ls # (alias: ollama list) — lists installed models
```

```
# Programmatic REST alternative: POST http://localhost:11434/api/generate (JSON)
```

- `ollama run <model> "<prompt>"` executes one prompt and prints to stdout (auto-pulls the model if absent). Output can be piped/redirected.
- `ollama ls / ollama list` enumerates installed models (machine-readable) — satisfies real model-listing for the Ollama-served subset.
- For richer JSON, the Ollama REST `/api/generate` endpoint returns JSON; HelixCode already integrates Ollama as a first-class provider (CONST-039), so these models should flow through the **existing Ollama provider path**, not a new per-agent CLI dispatch.

Operator decision (recommendation): 1. **Preferred:** Route the open-weight code models (StarCoder, CodeGemma, DeepSeek-Coder, WizardCoder, local Llama/Mistral code variants) through the **existing HelixCode Ollama provider** (`ollama run / Ollama REST + ollama ls` for discovery). This aligns with CONST-036/037/039 (LLMsVerifier as source of truth; Ollama already a supported provider) and avoids inventing fake per-agent CLIs. 2. **Provider-API path:** Route **Codey** → **Vertex/Gemini API** and **MistralCode** → **Mistral API** through HelixCode’s existing provider abstraction (CONST-039 lists Gemini + Mistral as required providers). 3. **Leave honest-error** for any of these ONLY as a per-agent *CLI dispatch* entry — i.e. the per-agent “CLI” stub stays honest-error stating “X is a model, not a CLI; use it via the

Ollama provider / provider API,” and the real capability is exposed through the provider layer, not the CLI-agent dispatch table.

Verdict: MODELS, not agents. No per-agent CLI. Real integration = Ollama provider (ollama run + ollama ls) for open-weight models; provider API for Codey/MistralCode. The CLI-dispatch stubs for these names should remain honest-error pointing at the provider path.

Final summary

Now have a CONFIRMED headless invocation (promote honest-error → real exec): 1. **Amazon Q** → **Kiro CLI** — kiro-cli chat --no-interactive "<prompt>" (+ KIRO_API_KEY, --trust-all-tools); models via kiro-cli chat --list-models --format json. (Prefer kiro-cli, fallback q.) 2. **Cody** — cody chat --stdin / -m '<prompt>' (+ --model); **caveat:** officially “human-interactive only / experimental,” and **no CLI model-listing** (list-models stays honest-error for Cody). 3. **SWE-agent** — sweagent run --problem_statement.text="<prompt>" --agent.model.name=<model>; result is a trajectory/patch (issue→patch harness), not chat text; no native list-models.

Confirmed NO headless CLI (stay honest-error): - **GPT-Pilot / Pythagora** — interactive by design, no headless flag. - **Devika** — web-UI + socket-driven; no documented single-prompt CLI/API (undocumented Flask routes = research-blocked, not CLI). - **Sourcegraph src** — not an LLM agent at all (code-search/admin CLI); miscategorized if listed as AI — remove or honest-error pointing to Cody.

Model group (LlamaCode/StarCoder/CodeGemma/DeepSeekCoder/MistralCode/Codey/WizardCoder): - These are **models, not CLIs**. **Recommendation:** wire the open-weight ones via the **existing Ollama provider** (ollama run <model> "<prompt>"), discover with ollama ls), and Codey/MistralCode via the **provider API** (Gemini/Mistral). Keep their per-agent CLI-dispatch entries as honest-error that redirect to the provider path. This respects CONST-036/037/039 and the no-fake-CLI posture.

Sources verified 2026-06-10

- Amazon Q Developer CLI — chat on the command line: <https://docs.aws.amazon.com/amazonq/latest/qdeveloper-ug/command-line-chat.html>
- Amazon Q Developer CLI — command reference: <https://docs.aws.amazon.com/amazonq/latest/qdeveloper-ug/command-line-reference.html>
- aws/amazon-q-developer-cli (repo — maintenance/Kiro migration notice): <https://github.com/aws/amazon-q-developer-cli>
- aws/amazon-q-developer-cli Issue #808 (Add Non-Interactive Mode): <https://github.com/aws/amazon-q-developer-cli/issues/808>
- Kiro CLI — Headless mode: <https://kiro.dev/docs/cli/headless/>
- Kiro CLI — CLI commands reference: <https://kiro.dev/docs/cli/reference/cli-commands/>
- Kiro CLI — Chat: <https://kiro.dev/docs/cli/chat/>
- Kiro — introducing headless mode (blog): <https://kiro.dev/blog/introducing-headless-mode/>
- Kiro Issue #5423 (machine-readable output for headless usage): <https://github.com/kirodotdev/Kiro/issues/5423>
- Cody — Install Cody CLI: <https://sourcegraph.com/docs/cody/clients/install-cli>
- Cody — docs root: <https://sourcegraph.com/docs/cody>

- Cody VS Code 1.16 / models (blog): <https://sourcegraph.com/blog/cody-vscode-1-16-0-release>
- @sourcegraph/cody (npm): <https://www.npmjs.com/package/@sourcegraph/cody>
- Sourcegraph src-cli (repo): <https://github.com/sourcegraph/src-cli>
- Sourcegraph CLI docs: <https://sourcegraph.com/docs/cli>
- SWE-agent — CLI usage: <https://swe-agent.com/latest/usage/cli/>
- SWE-agent — command line basics: https://swe-agent.com/latest/usage/cl_tutorial/
- mini-swe-agent (repo): <https://github.com/SWE-agent/mini-swe-agent>
- GPT-Pilot (repo): <https://github.com/Pythagora-io/gpt-pilot>
- GPT-Pilot (PyPI): <https://pypi.org/project/gpt-pilot/>
- GPT-Pilot Wiki (FAQ / usage): <https://github.com/Pythagora-io/gpt-pilot/wiki>
- Devika (repo): <https://github.com/stitionai/devika>
- Devika README: <https://github.com/stitionai/devika/blob/main/README.md>
- Devika ARCHITECTURE: <https://github.com/stitionai/devika/blob/main/ARCHITECTURE.md>
- Devika devika.py (Flask backend): <https://github.com/stitionai/devika/blob/main/devika.py>
- Ollama — CLI reference: <https://docs.ollama.com/cli>
- Ollama — CLI (DeepWiki): <https://deepwiki.com/ollama/ollama/1.3-command-line-interface>
- Ollama README: <https://github.com/ollama/ollama/blob/main/README.md>

Negative findings (explicit): No JSON-output flag or model-listing subcommand found for Cody CLI; no native list-models for SWE-agent (delegates to LiteLLM); no documented headless flag for GPT-Pilot; no documented single-prompt CLI/HTTP endpoint for Devika; `src` CLI has no LLM/AI capability. Kiro CLI is closed-source (the open-source Amazon Q CLI is in security-only maintenance).